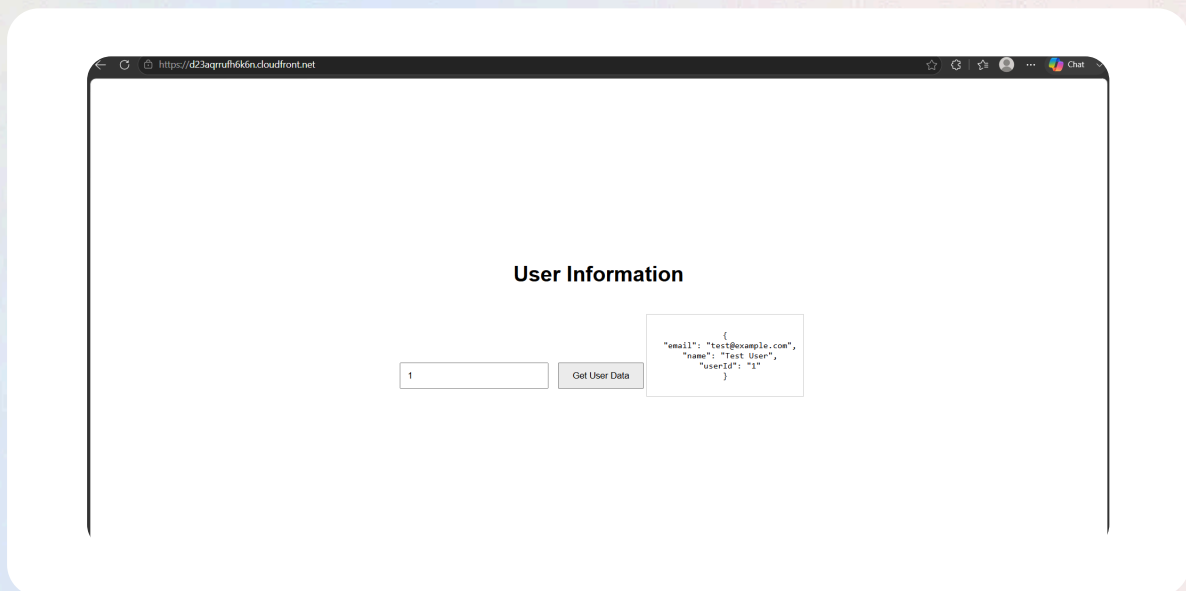


BUILD A THREE-TIER WEB APP

By Mohd Mudassir Arfath | July 2026

Build a Three-Tier Web App



Introducing Today's Project!

We are here to build a three-tier serverless web application on AWS by connecting S3, CloudFront, API Gateway, Lambda, and DynamoDB together. This will help us learn how these AWS services work together to create a scalable and reliable application.

Tools and concepts

I learned about key AWS services such as Amazon S3, CloudFront, API Gateway, AWS Lambda, and DynamoDB. I also learned how these services work together in a serverless three-tier web application, along with important concepts like CORS, API integration, IAM permissions, CloudFront caching, and secure access to AWS resources.

Project reflection

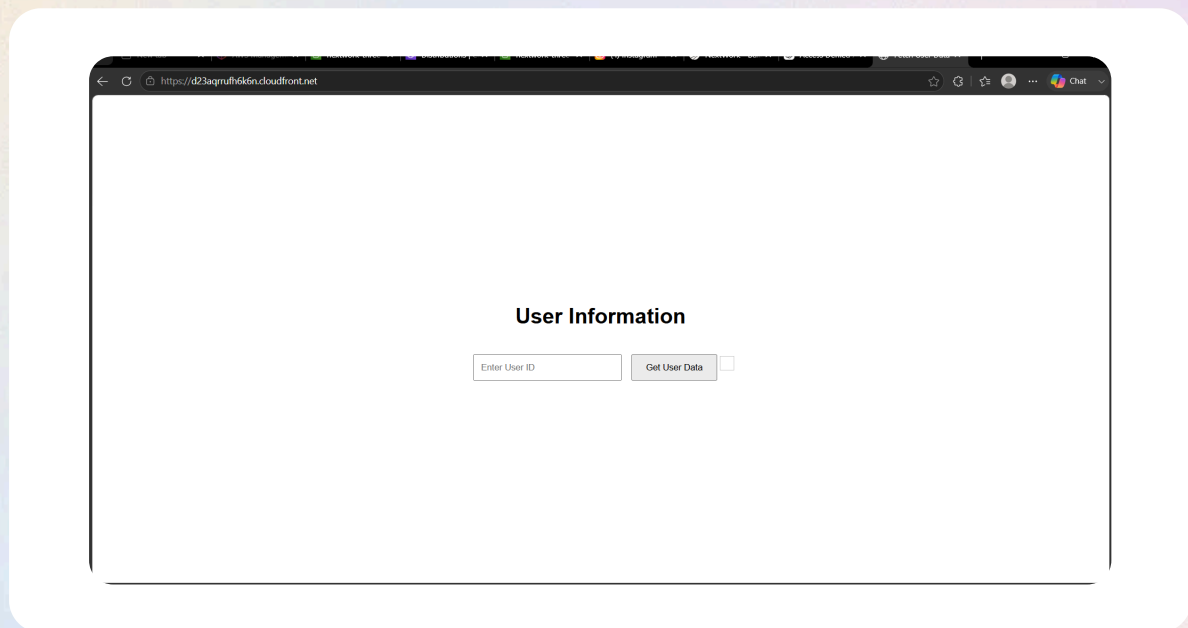
It took me approximately 3–4 hours to complete this project, including configuring the AWS services, troubleshooting errors, and testing the final solution.

Thank you! I enjoyed working on this project and learned a lot about building and troubleshooting a serverless web application using AWS services.

Presentation tier

We are setting up the presentation tier, which is the front-end of our application. We will use Amazon S3 to store the website files and CloudFront to deliver the website content quickly to users around the world.

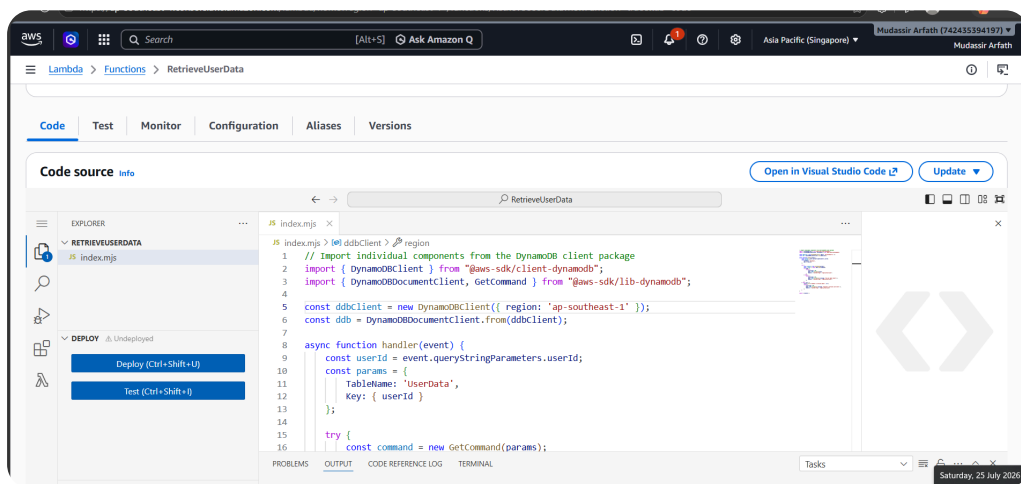
I accessed my website using the CloudFront distribution URL in a web browser. CloudFront delivered the website content from my S3 bucket to the browser.



Logic tier

We are setting up the logic tier by creating an AWS Lambda function that retrieves data from a DynamoDB table. We are also using API Gateway to create a REST API that handles GET requests and connects users to the Lambda function.

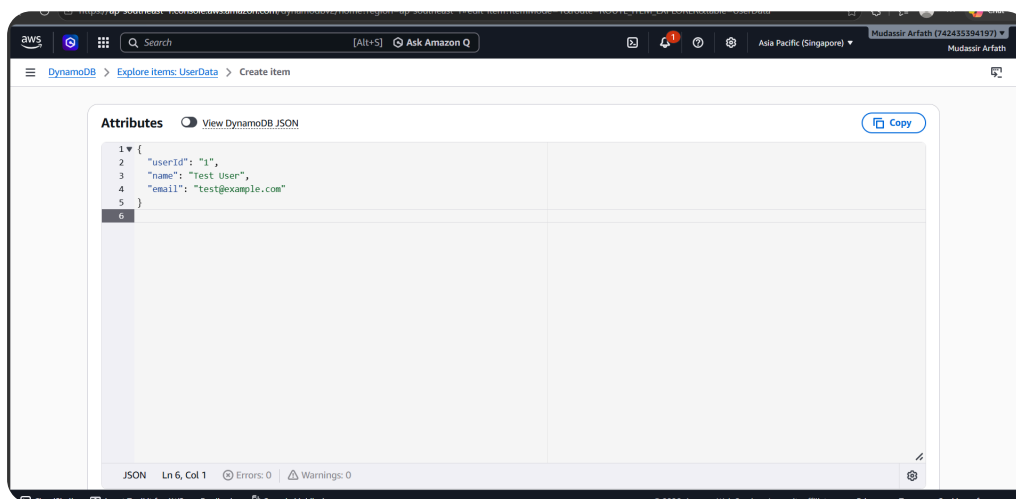
The Lambda function retrieves data by using the AWS SDK to connect to the DynamoDB table. It sends a Scan or Query request to DynamoDB, retrieves the stored items, and returns the data as the response.



Data tier

We are setting up the data tier by creating an Amazon DynamoDB table to store application data. We will also add user data to the table so that the Lambda function can retrieve it when requested.

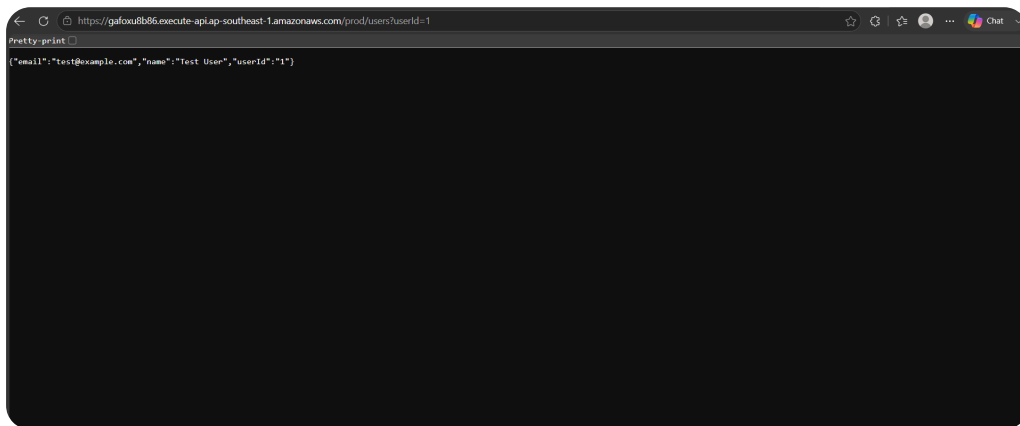
We are using DynamoDB to store and manage user data for our application. The Lambda function will later retrieve this data from the DynamoDB table when a user makes a request...



Logic and Data tier

We are integrating all three tiers of our application. We will update the script.js file to send a request to the API Gateway endpoint, receive the data retrieved from DynamoDB through Lambda, and display that data on our website.

I tested my API by sending a GET request to the API Gateway endpoint in a web browser. I verified that the API successfully returned the user data retrieved from the DynamoDB table through the Lambda function.

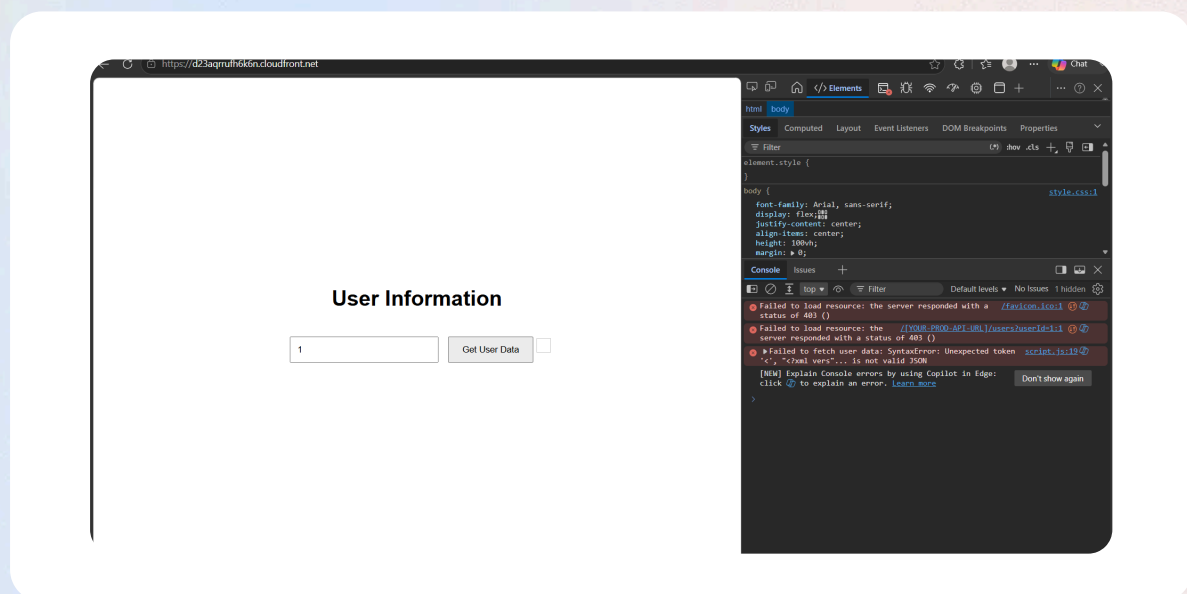


Console Errors

I got an error because the script.js file was still using the placeholder YOUR-PROD-API-URL instead of the actual API Gateway endpoint URL. Therefore, the CloudFront website could not connect to the API Gateway and retrieve the user data.

I uploaded an updated script.js because the original file contained a placeholder for the API Gateway URL. I replaced the placeholder with the actual production API Gateway Invoke URL so the CloudFront website could connect to the API and retrieve user data.

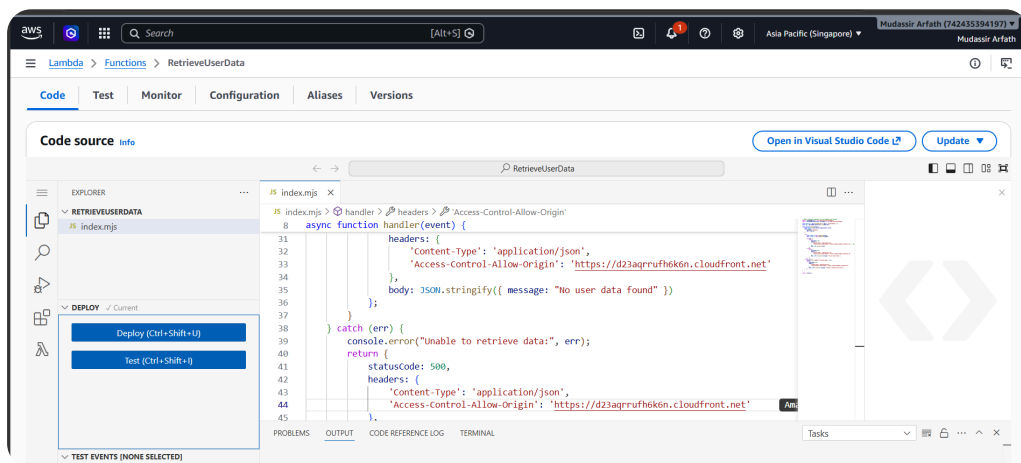
I ran into another error because the browser blocked the request from the CloudFront website to the API Gateway due to a CORS (Cross-Origin Resource Sharing) issue. The API needs to allow requests coming from the CloudFront website's domain.



Resolving CORS Errors

I updated the CORS configuration in my API Gateway. I allowed GET and OPTIONS methods and set my CloudFront distribution domain as the allowed origin, so my CloudFront website can securely make requests to the API.

I updated my Lambda function because the CORS headers need to be included in the Lambda response. This allows the browser to accept responses from the API when requests are made by my CloudFront website.



Fixed Solution

I verified the fixed solution by refreshing my CloudFront website and checking that the data from DynamoDB was successfully fetched and displayed on the webpage without any CORS errors.

